

Ingeniería del Software de Componentes y Sistemas Distribuidos

SEGUNDA PRUEBA PRÁCTICA – PRAC 2

Presentación

En la PRAC 1 se ha diseñado una solución para el caso **Photo&Film4You** basada en microservicios, usando modelos de representación y patrones de diseño. En esta PRAC 2 se pide implementar un subconjunto de este diseño a partir de programar 3 microservicios considerando diferentes aspectos, como, por ejemplo: estado propio, comunicación síncrona, lógica de negocio, consistencia de los datos y despliegue en local.

Para elaborar esta práctica se formulan 2 ejercicios a partir del caso **Photo&Film4You**:

- Implementar tres de los microservicios diseñados en la PRAC1, según un conjunto de consideraciones técnicas y la documentación de las evidencias.
- Razonar cómo resolver la gestión de transacciones en una arquitectura distribuida en microservicios.

La PRAC 2 abarca los contenidos del libro "Microservices Patterns" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta PRAC 2 se trabaja las siguientes competencias del Grado en Ingeniería Informática:

- Saber construir aplicaciones informáticas mediante técnicas de desarrollo, integración y reutilización.
- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.

Objetivos

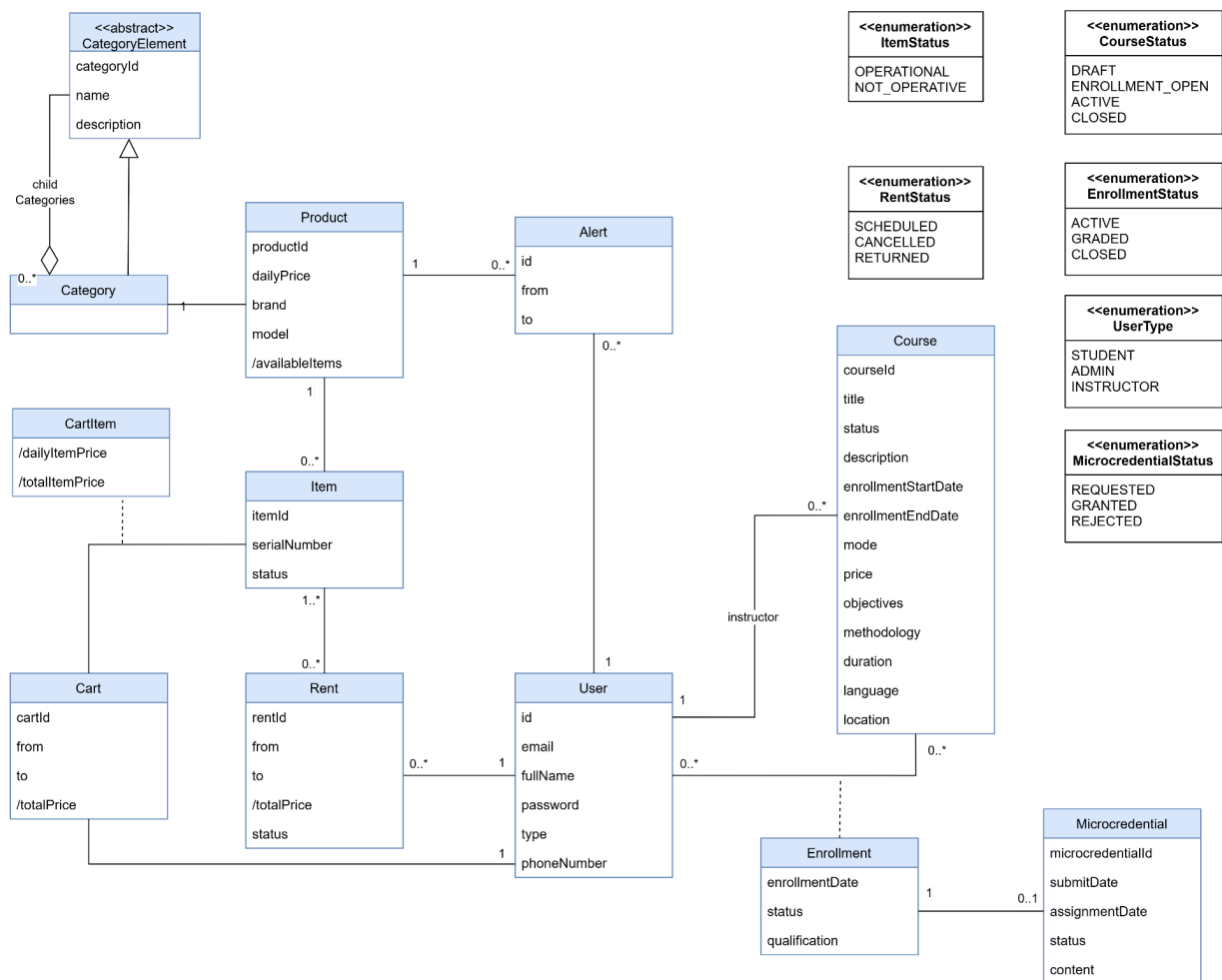
Los objetivos de la PRAC 2 son:

- Implementar microservicios centrados en la lógica de negocio y el acceso a datos.
- Desplegar microservicios con estado propio e intercomunicación síncrona o asíncrona.
- Aplicar conceptos básicos de la arquitectura de microservicios.
- Dominar las tecnologías básicas necesarias para la implementación de microservicios.

Descripción de la PRAC 2

Ejercicio 1 (8 puntos)

Este primer ejercicio consiste en implementar un subconjunto de los microservicios que se diseñaron en la PRAC1. Esta parte debe basarse en la solución publicada de la PRAC1, incluyendo el siguiente diagrama del modelo de dominio general del cual se pueden extraer las principales entidades (con sus atributos) y relaciones:



El subconjunto de servicios a implementar se compone de los servicios Course, Notification y Microcredential.

Los microservicios que tendrá el proyecto serán los siguientes:

- El microservicio *Product Catalog* servirá tanto para dar cobertura a las diversas operaciones que los administradores de la plataforma necesitan para gestionar el inventario (crear/eliminar productos, categorías/subcategorías y unidades de producto) como para las consultas del catálogo de productos que realizarán los usuarios.
- El microservicio *User* sirve para gestionar las altas, los datos personales de los usuarios, y la configuración de alertas, los contenidos digitales y sus elementos digitales. Los usuarios deben poder dar de alta alertas para un producto concreto y un rango de fechas. Cuando una unidad de ese producto quede libre para este rango de fechas, debe emitirse un mensaje a la cola para que otro servicio se encargue de hacer efectiva la alerta (microservicio *Notification*).
- El microservicio *Course* es responsable de gestionar los cursos, las inscripciones y la gestión académica básica.
- El microservicio *Microcredential* es responsable de gestionar la generación y aprobación de credenciales para cursos completados y calificados por alumnos.
- El microservicio *Notification* tiene la finalidad de emitir notificaciones a los usuarios interesados en un producto concreto, cuando se añaden unidades o alguna queda libre para ser alquilada. Este microservicio debe escuchar la cola de mensajes de forma que cuando se libere una unidad de producto para una fecha en la que algún usuario tenga configurada una alerta, a este usuario se le mande un correo electrónico con el aviso. También a los administradores cuando haya una microcredencial pendiente de aceptar o a los estudiantes cuando haya una microcredencial generada.

De acuerdo con lo descrito, estos cuatro microservicios tendrán como mínimo las siguientes operaciones (implementar solo los que no estén implementados):

Microservicio Course (gestión de cursos):

- alta de curso
- modificar detalles del curso
- abrir inscripción
- cerrar inscripción
- inscribirse en curso
- cerrar acta de calificaciones
- cerrar curso
- buscar cursos
- consulta curso por identificador
- consulta de los estudiantes inscritos
- consulta inscripciones por curso

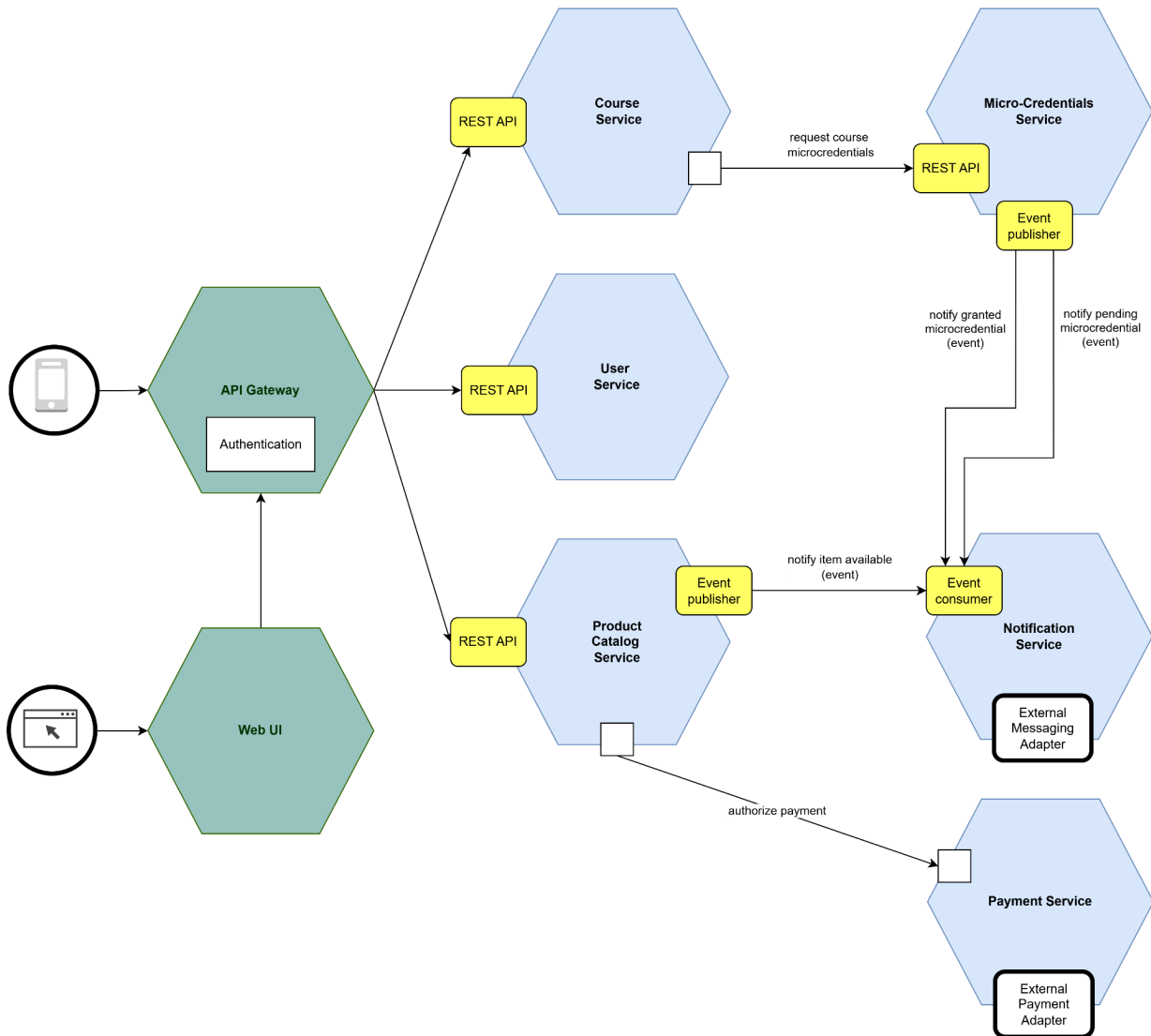
Microservicio Microcredencial (gestión de microcredenciales):

- pedir microcredencial
- aprobar microcredencial pendiente
- rechazar microcredencial pendiente
- solicitar microcredencial de curso

Microservicio Notification (notificaciones automáticas a usuarios):

- enviar notificación microcredencial pendiente
- enviar notificación microcredencial obtenida

En el siguiente diagrama se puede ver cómo debe ser la colaboración entre los diversos servicios del sistema:



Consideraciones previas

Para poder conseguir la flexibilidad, resiliencia y rendimiento que caracteriza esta arquitectura, es necesario cumplir una serie de requerimientos:

- Cada microservicio:
 - tiene que poder ejecutarse independientemente,
 - tiene que tener estado propio (y acceder de manera independiente a la BBDD),
 - implementa sólo su lógica de negocio,
 - expone su contrato (definición de la API),
 - gestiona su propio *log*.
- Para ciertas operaciones, puede ser necesaria la colaboración entre varios microservicios para obtener información. En este caso, los microservicios pueden realizar peticiones a otros microservicios por medio de llamadas REST.
- Sólo es necesario implementar las operaciones de los microservicios *Notification*, *Course* y *Microcredential*. Las operaciones de los demás microservicios ya están implementadas.
- Cosas que quedan fuera del alcance del proyecto en la fase piloto que se pide implementar:
 - Autenticación: no será necesario ningún tipo de autenticación para ninguna de las operaciones, se da por hecho que esta parte de la funcionalidad del sistema la proporciona el API Gateway (que no debe implementarse).
 - Service Discovery & Registration: no se debe implementar ningún tipo de funcionalidad de registro y búsqueda de servicios, se da por hecho que sólo existe una única instancia, en una ubicación conocida.
 - Envío de correos: para no tener que implementar esta parte, de poco valor didáctico dentro de la PRAC, os sugerimos que simuléis su comportamiento con el uso de Mocks. Se debe poder demostrar al menos una línea de log cada vez que se envíe una notificación (por ejemplo, mostrar mediante el log del microservicio *Notification* una línea de log con nivel INFO que diga “Se ha notificado por correo el usuario XXX YYY ZZZ que existe una unidad disponible del producto AAA”).

Plataforma de desarrollo

- Entorno de desarrollo: IntelliJ IDEA / Eclipse
- JDK: OpenJDK 25
- Framework: Spring / Spring Boot
- Gestión de dependencias y build: Maven
- Servidor HTTP: Tomcat (integrado por defecto con Spring Boot)
- Librería llamadas REST: Jersey
- Comunicación ASÍNCRONA entre servicios: Apache Kafka
- BBDD: PostgreSQL
- Implementación ORM: Hibernate
- Generación de la documentación del API OpenAPI v3: springdoc-openapi
- Log: Log4j2
- Mocks: Mockneat (para generar datos pseudo-aleatorios)

Punto de partida y recursos

Para tener un punto de partida común y homogéneo, y poder centrar los esfuerzos en el desarrollo de los microservicios, junto con este enunciado se proporciona un enlace a un repositorio público (<https://github.com/UOC-SPRING-2026>) con los siguientes recursos:

- Un archivo *docker-compose.yml* para instalar la infraestructura básica para la implementación de este ejercicio y un archivo *README.md* con instrucciones para arrancarla.
- Esqueletos de aplicación Spring Boot con las funcionalidades básicas para generar:
 - un archivo java jar ejecutable con todas las dependencias contenidas en el proyecto (para poder hacer debug localmente);
 - un archivo de imagen docker con el microservicio y todas sus dependencias (para poder hacer la entrega).
- Un conjunto de enlaces a documentación, guías y tutoriales relativos tanto a las herramientas a utilizar como a la propia implementación de los microservicios solicitados (se colgarán en el aula de Laboratorio).

Cada uno de los esqueletos de aplicación que se proporcionan son un proyecto Spring Boot completo pre-configurado con las dependencias mínimas necesarias para hacer toda la implementación que se pide en este enunciado. Esto no significa que no se puedan añadir otras dependencias si os parece adecuado, pero con lo que hay basta para hacer toda la implementación.

Con los contenedores del archivo *docker-compose* arrancados, todos los esqueletos de microservicio tendrían que poder ponerse en marcha y conectar a la infraestructura tal como se entregan con esta PRAC2. Además, aprovechando que los esqueletos llevan su propio servidor HTTP empotrado, se ha incluido un módulo *springdoc-openapi* que auto-genera y publica el archivo

de definición del microservicio (API) en formato *JSON/OpenAPI v3*, así como una pequeña interfaz web *SwaggerUI* que permite hacer llamadas a las operaciones de los microservicios.

Las direcciones para acceder a estas funcionalidades son:

- **archivos de definición de los servicios:**
 - Product Catalog: <http://localhost:18081/v3/api-docs/>
 - User: <http://localhost:18082/v3/api-docs/>
 - Course: http://localhost:18084/v3/api-docs
 - Microcredential: http://localhost:18085/v3/api-docs
- **interfaces web SwaggerUI para pruebas:**
 - Product Catalog: <http://localhost:18081/swagger-ui/index.html>
 - User: <http://localhost:18082/swagger-ui/index.html>
 - Course: <http://localhost:18084/swagger-ui/index.html>
 - Microcredential: <http://localhost:18085/swagger-ui/index.html>

Cómo se puede ver, estos ejemplos forman una base bastante completa desde donde empezar el desarrollo y, por lo tanto, se tendría que evitar hacer modificaciones en sus componentes básicos (excepto pequeñas configuraciones, etc.), y solo añadir las funcionalidades que se piden en este enunciado. Encontraréis más información y apoyo técnico en el Laboratorio de la asignatura.

Entregable

Especificaciones técnicas de la aplicación a entregar

El entregable de esta parte del ejercicio será un único archivo ZIP que tiene que contener:

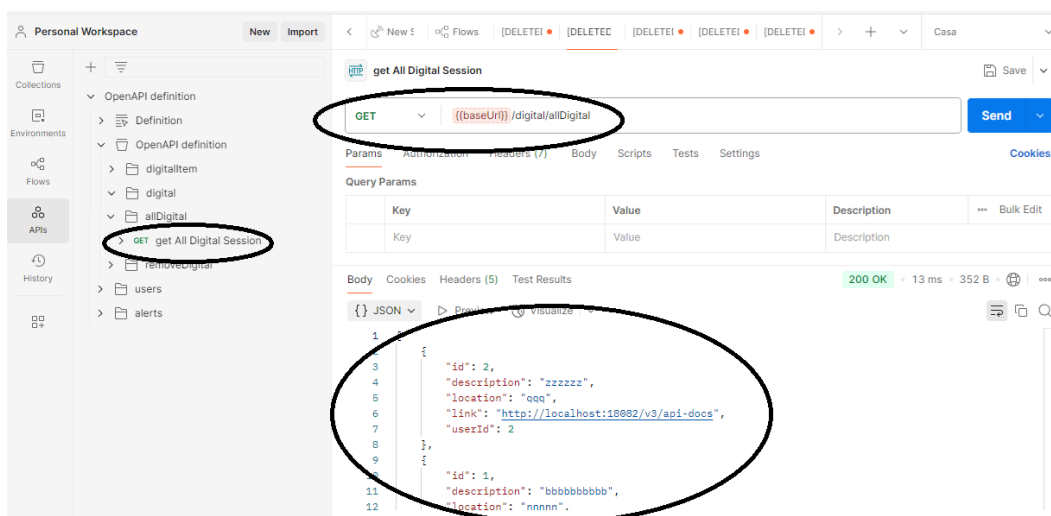
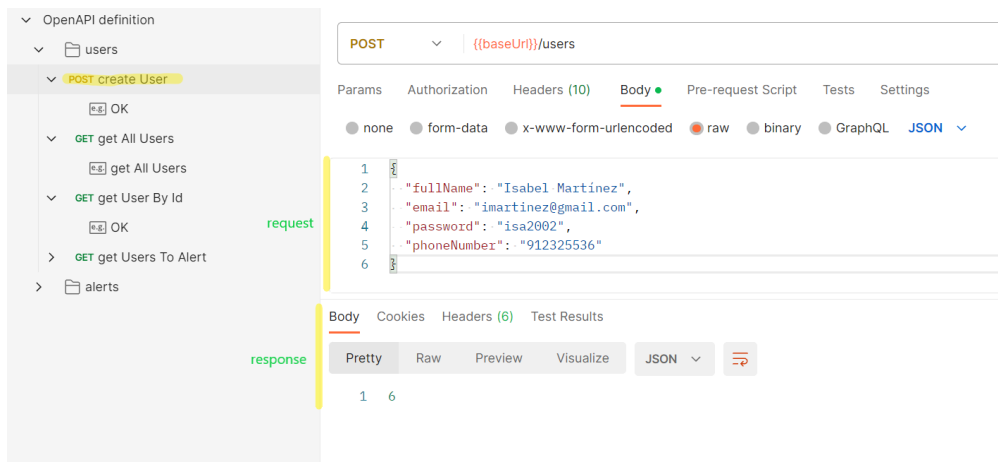
- Todo el código y archivos de configuración del proyecto en un **archivo ZIP** (dentro del ZIP “principal”).
- Un **archivo ZIP** que contiene el código fuente del proyecto.
- Un archivo **readme.txt** con la siguiente información:
 - cualquier configuración necesaria para poder ejecutar los servicios;
- Un **PDF** con capturas de pantalla significativas que demuestren la funcionalidad del proyecto.

Ejercicio 2 (2 puntos)

Para poder demostrar que se han logrado los objetivos de la implementación, tendréis que proporcionar evidencias del correcto funcionamiento de los microservicios. Para elaborar estas evidencias os proponemos utilizar la aplicación Postman, una herramienta que os permitirá importar el archivo de definición del servicio (accesible desde la ruta v3/api-docs de los tres microservicios) y generar una colección que contendrá todas las operaciones disponibles. Como mínimo se realizará un ejemplo de cada tipo de llamada con datos “reales” una vez puestos en marcha los microservicios.

Entregable

El entregable de esta parte del ejercicio será una captura de pantalla con la llamada y resultado para cada una de las operaciones implementadas en la primera parte de los **microservicios Course y Microcredential**.



Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson): Chapter 6: "Developing business logic with event sourcing" (excepto sección 6.3).
- Libro "Microservices patterns" (Richardson): Chapter 7: "Implementing queries in a microservice architecture"
- Libro "Microservices patterns" (Richardson): Chapter 8: "External API patterns" (excepto sección 8.3).

Recursos Complementarios

- Repositorio de código fuente del ejemplo del libro Microservices Patterns.
<https://github.com/microservices-patterns/ftgo-application>
- Ejemplo de programación de microservicios.
<https://github.com/vmudigal/microservices-sample>

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. Esto incluye la reutilización de soluciones de esta práctica de semestres anteriores. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Los criterios de evaluación de cada ejercicio están indicados en el anexo.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Debe entregarse un único archivo ZIP con las respuestas a los dos ejercicios. El nombre del archivo debe ser: *ApellidosNombre_ISCSD_PRAC2.zip* con la siguiente estructura:

- ZIP con la entrega del ejercicio 1, que debe contener:
 - ZIP con el código fuente
 - un archivo JAR ejecutable por cada microservicio implementado

- un archivo `readme.txt` con instrucciones adicionales para arrancar la infraestructura y/o microservicios (especialmente en caso de cambios en la configuración o en el `docker-compose`)
- Capturas de pantalla de Postman del ejercicio 2.

Este documento se entregará en el espacio “Entrega de la PRAC2” del aula antes de las **23:59 horas del día 18 de mayo de 2026**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación

	Excelente (10)	Notable (7)	Suficiente (5)	Insuficiente (2,5)	Sin respuesta / Todo mal (0)
Ej. 1	• Número de operaciones no presentes en el esqueleto implementadas correctamente desde el punto de vista funcional (con 10 como máxima puntuación) • Uso correcto de los verbos HTTP • Control de errores (IDs que no existen, errores de tipo, etc.), validaciones y retorno del estado HTTP adecuado al resultado de la llamada • Colaboración completa entre servicios para llamadas REST • Utilizar todos los parámetros query/path según las	• Número de operaciones no presentes en el esqueleto implementadas correctamente desde el punto de vista funcional (mínimo 8) • Uso generalmente correcto de los verbos HTTP • Control de errores (IDs que no existen, errores de tipo, etc.), validaciones y retorno del estado HTTP generalmente adecuado al resultado de la llamada • Algunos problemas con la colaboración entre servicios para llamadas REST • Utilizar la mayoría de parámetros query/path	• Número de operaciones no presentes en el esqueleto implementadas correctamente desde el punto de vista funcional (mínimo 5) • Uso generalmente incorrecto de los verbos HTTP • Control de errores (IDs que no existen, errores de tipo, etc.), validaciones y retorno del estado HTTP generalmente inadecuado al resultado de la llamada • Muchos problemas con la colaboración entre servicios para llamadas REST • Uso de parámetros query/path, pero no siguen las	• Número de operaciones no presentes en el esqueleto implementadas correctamente desde el punto de vista funcional (menos de 5) • Uso incorrecto de los verbos HTTP • Control de errores (IDs que no existen, errores de tipo, etc.), validaciones y retorno del estado HTTP no adecuado al resultado de la llamada • No hay colaboración entre servicios para llamadas REST • No uso de parámetros query/path y/o no siguen las	• Sin respuesta, no hay ninguna justificación o todo incorrecto (0)

	convenciones estándar • Utilizar DTOs para agrupar inputs/outputs • Justificación/explicación correcta en las partes donde sea necesario. • Calidad máxima de la implementación (legibilidad, formateo, sin warnings, etc.) (8)	según las convenciones estándar • Utilizar DTOs para agrupar inputs/outputs • Justificación/explicación incorrecta o en lugares donde no es necesaria. • Calidad general aceptable de la implementación (legibilidad, formateo, sin warnings, etc.) (5,5)	convenciones estándar • No utilización de DTOs para agrupar inputs/outputs • Ausencia de justificación/explicación. • Calidad general pobre de la implementación (legibilidad, formateo, sin warnings, etc.) (4)	convenciones estándar • No utilización de DTOs • Calidad inaceptable de la implementación (legibilidad, formateo, sin warnings, etc.) (2)	
Ej. 2	Evidencias para todas las operaciones correctas, con información (entrada y salida) concreta para cada una de ellas (2)	Evidencias para aproximadamente 3/4 de las operaciones correctas, con información (entrada y salida) concreta para cada una de ellas. (1,5)	Evidencias para aproximadamente la mitad de las operaciones correctas, con información (entrada y salida) concreta para cada una de ellas. (1)	Evidencias para aproximadamente menos de la mitad de las operaciones correctas, con información (entrada y salida) concreta para cada una de ellas. (0,5)	Sin respuesta, no hay ninguna operación adicional respecto a las que ya tenía el esqueleto o no se puede importar el archivo de definición (0)